📘 GUIA COMPLETO - RW JOIAS LANDING PAGE

Bem-vindo ao guia oficial do projeto **RW Joias**. Este documento foi cuidadosamente reestruturado para ser um material de estudo didático, ideal para quem está aprendendo desenvolvimento web moderno com Vite e Tailwind CSS e busca uma vaga como Desenvolvedor Júnior.

📑 SUMÁRIO

1. Visão Geral do Projeto
2. Pré-requisitos e Instalação
   - 2.1. Verificar Node.js
   - 2.2. Instalar Dependências
   - 2.3. Rodar o Servidor de Desenvolvimento
3. Estrutura do Projeto
4. Configuração do Vite
5. Configuração do Tailwind CSS e PostCSS
   - 5.1. O Arquivo `postcss.config.js`
   - 5.2. O Arquivo `tailwind.config.js`
   - 5.3. O CSS Organizado (`src/style.css`)
6. Guia de Cores e Design
   - 6.1. Paleta de Cores
   - 6.2. Tipografia e Fontes
   - 6.3. Hierarquia Tipográfica
   - 6.4. Boas Práticas de Acessibilidade
7. Anatomia do Código (HTML, CSS e JS)
   - 7.1. O HTML Limpo (`index.html`)
   - 7.2. O JavaScript Modular (`src/main.js`)
   - 7.3. Funções de Interatividade
8. Como Editar e Personalizar
   - 8.1. Trocar Imagens
   - 8.2. Editar Textos
   - 8.3. Adicionar Nova Coleção
   - 8.4. Mudar Cores para Outros Negócios
9. GitHub e Versionamento
   - 9.1. Fluxo Básico do Git
   - 9.2. Convenção de Commits
10. Deploy - Colocando Online
    - 10.1. GitHub Pages
    - 10.2. Netlify (Gratuito)
    - 10.3. Vercel (Gratuito)
11. 🛠️ Onde Errei e o Que Foi Melhorado (Análise de Erros)
    - 11.1. Erro de CSS (PostCSS)
    - 11.2. Importação do CSS no HTML
    - 11.3. Funções de Modal não Acessíveis (Scope Issue)

    - 11.4. Configuração Incompleta do Tailwind v4
    - 11.5. Ordem dos Imports no CSS
    - 11.6. Melhorias Visuais Adicionais
12. Próximos Passos e Ideias para Evoluir

---

1. VISÃO GERAL DO PROJETO

O projeto **RW Joias** é uma landing page premium desenvolvida para uma joalheria fictícia. O objetivo principal é demonstrar a construção de um site moderno e responsivo utilizando as tecnologias mais atuais do ecossistema JavaScript, com foco em performance e experiência do usuário. Este projeto serve como um excelente ponto de partida para quem busca aprimorar suas habilidades em desenvolvimento front-end.

Principais Características:

- **Design Luxo:** Paleta de cores sofisticada baseada em preto, dourado e branco, transmitindo elegância e exclusividade.
- **Animações e Interatividade:** Implementação de efeitos de *scroll reveal*, animações de *hover* em cards, modais expansíveis para visualização de coleções e um menu mobile funcional.
- **Responsividade:** Desenvolvido com uma abordagem *mobile-first*, garantindo que o layout e a funcionalidade se adaptem perfeitamente a qualquer tamanho de tela, desde smartphones até desktops de alta resolução.
- **Performance Otimizada:** Utilização do **Vite** como ferramenta de build, proporcionando um ambiente de desenvolvimento extremamente rápido com *Hot Module Replacement (HMR)* e builds otimizados para produção.
- **Estilização Eficiente:** Uso do **Tailwind CSS** para estilização rápida e consistente, permitindo a criação de designs complexos diretamente no HTML com classes utilitárias.

Tecnologias Utilizadas:

Tecnologia	Versão	Função Principal
Vite	v8.0+	Ferramenta de build de próxima geração e servidor de desenvolvimento local.
Tailwind CSS	v4.2+	Framework CSS utilitário para estilização rápida e responsiva.
PostCSS	v8.5+	Ferramenta para transformar CSS com plugins, essencial para o processamento do Tailwind.
JavaScript	Vanilla	Implementação da lógica de interatividade, como modais, menu mobile e animações de scroll.
HTML5	-	Estruturação semântica e acessível do conteúdo da página.

---

2. PRÉ-REQUISITOS E INSTALAÇÃO (PRIMEIRA VEZ)

Para rodar este projeto localmente, estudar seu código e fazer suas próprias modificações, você precisará ter o Node.js e Git instalado em sua máquina. O Node.js inclui o npm (Node Package Manager), que será usado para gerenciar as dependências do projeto.

2.1. Verificar Node.js e Git

Abra o terminal (ou o terminal integrado do VS Code) e execute os seguintes comandos para verificar se o Node.js e o npm estão instalados:

```
node --version
npm --version
git --version
```

Se os comandos retornarem números de versão (ex: `v20.11.0` para Node.js e `10.2.4` para npm), você está pronto para prosseguir. Caso contrário, baixe e instale a versão LTS (Long Term Support) mais recente do Node.js.

2.2. Instalar Dependências

Com o terminal aberto na pasta raiz do projeto (`rw-joias`), execute o comando abaixo para baixar e instalar todas as bibliotecas e ferramentas necessárias (Vite, Tailwind CSS, PostCSS, Autoprefixer, etc.):

```
npm install
```

Este comando lerá o arquivo `package.json` e instalará todas as dependências listadas em `devDependencies` na pasta `node_modules/`.

2.3. Rodar o Servidor de Desenvolvimento

Após a instalação das dependências, você pode iniciar o servidor de desenvolvimento local do Vite. Isso permitirá que você visualize o projeto no navegador e veja as alterações em tempo real enquanto codifica.

```
npm run dev
```

O terminal exibirá uma mensagem indicando que o servidor está rodando, geralmente com um link como `http://localhost:5173/rw-joias/`. Copie e cole este link no seu navegador para abrir o projeto.

> **Dica:** O Vite oferece *Hot Module Replacement (HMR)*, o que significa que a maioria das suas alterações no código (HTML, CSS, JavaScript) serão refletidas instantaneamente no navegador, sem a necessidade de recarregar a página manualmente.

2.3 Clonando projeto ja no GIT

Clone o repositório (substitua com seu usuário)

git clone https://github.com/SEU-USUARIO/NOME-REPOSITORIO.git

Entre na pasta do projeto

cd rwjoias

Instala todas as bibliotecas (Vite, Tailwind, etc.)

npm install

Inicia o servidor local com Hot Reload

npm run dev

Acesse no navegador: http://localhost:5173/rwjoias/

2.4 Fluxo de Trabalho Diário

Antes de começar a codar:

git pull origin main # Puxa alterações mais recentes

Depois de fazer alterações:

git add . # Prepara arquivos git commit -m "feat: sua mensagem" # Salva alterações git push origin main # Envia para o GitHub

📋 Resumo Rápido - Comandos Essenciais

🔄 Atualizar projeto local

git pull origin main

📦 Instalar dependências (se node_modules sumir)

npm install

🚀 Rodar em desenvolvimento

npm run dev

🏗️ Build para produção

npm run build

🌐 Deploy para GitHub Pages

npm run deploy

⚠️ Nunca edite a pasta node_modules/. Se algo quebrar, delete a pasta e rode npm install novamente.

🦠 Problemas Comuns no Setup Problema npm install demora muito Solução Tente npm install --legacy-peer-deps

Problema Erro de permissão no Git Solução Configure: git config --global user.name "Seu Nome"

Problema Porta 5173 ocupada Solução Vite usa outra porta automaticamente, ou use npm run dev -- --port 3000

Problema CSS não atualiza Solução Limpe cache do navegador: Ctrl+Shift+R

Problema Imagens não carregam no deploy Solução Use paths absolutos: /images/foto.jpg

✅ Checklist - Projeto Pronto para Codar git clone executado com sucesso npm install concluído sem erros npm run dev rodando e site acessível no navegador Alterações no HTML/CSS/JS refletem em tempo real (HMR) git pull e git push funcionando 💡 Dica: Mantenha sempre seu repositório atualizado com git pull antes de começar a trabalhar, para evitar conflitos com outras alterações.

---

3. ESTRUTURA DO PROJETO

Compreender a organização das pastas e arquivos é um passo crucial para qualquer desenvolvedor. A estrutura a seguir é padrão para projetos Vite e ajuda a manter o código organizado e escalável.

```
rw-joias/
├── node_modules/           # Contém todas as bibliotecas e dependências
instaladas pelo npm. **Não deve ser versionado no Git.**
├── public/                 # Armazena assets estáticos que são servidos
diretamente, como `favicon.svg` e `icons.svg`. Imagens grandes também podem ir
aqui.
├── src/                    # Contém o código-fonte principal da sua aplicação.
│   ├── main.js             # O arquivo JavaScript principal. É o ponto de entrada
da aplicação, onde o CSS é importado e a lógica JS é inicializada.
│   └── style.css           # O arquivo CSS principal, onde as diretivas do
Tailwind e seus estilos customizados são definidos.
├── index.html              # O arquivo HTML principal. É a única página do
projeto e o ponto de entrada para o navegador.
├── package.json            # Define as informações do projeto, scripts de
execução (dev, build, deploy) e lista todas as dependências.
├── postcss.config.js       # Arquivo de configuração para o PostCSS, que processa
o CSS, incluindo o Tailwind CSS e o Autoprefixer.
├── tailwind.config.js      # Arquivo de configuração do Tailwind CSS, onde você
personaliza o tema, cores, fontes, breakpoints, etc.
├── vite.config.js          # Arquivo de configuração do Vite, usado para definir
```

```
o comportamento do servidor de desenvolvimento e do processo de build.
└── README.md               # Este documento, contendo toda a documentação do
projeto.
```

---

4. CONFIGURAÇÃO DO VITE

O Vite é uma ferramenta de build moderna que oferece uma experiência de desenvolvimento extremamente rápida. A configuração do Vite é feita através do arquivo `vite.config.js`.

Arquivo `vite.config.js`:

```
import { defineConfig } from 'vite'

export default defineConfig({
  base: '/rw-joias/',  // IMPORTANTE: Define o caminho base para o deploy em
subdiretórios (ex: GitHub Pages)
  build: {
    outDir: 'dist',    // Pasta onde os arquivos otimizados para produção serão
gerados
    assetsDir: 'assets', // Subpasta dentro de `outDir` para assets (CSS, JS,
imagens)
  }
})
```

Explicação:

- `base: '/rw-joias/'`: Esta propriedade é crucial para projetos que serão hospedados em subdiretórios, como o GitHub Pages. Se o seu repositório se chama `rw-joias`, o site será acessível em `https://seunome.github.io/rw-joias/`. O `base` garante que todos os caminhos para seus assets (CSS, JS, imagens) sejam gerados corretamente, evitando links quebrados.
- `build.outDir: 'dist'`: Define que todos os arquivos de produção (HTML, CSS, JS otimizados) serão colocados na pasta `dist/` após o comando `npm run build`.
- `build.assetsDir: 'assets'`: Especifica que os assets (CSS, JS, imagens) dentro da pasta `dist/` serão organizados em uma subpasta chamada `assets/`.

---

5. CONFIGURAÇÃO DO TAILWIND CSS E POSTCSS

O Tailwind CSS é um framework utilitário que permite estilizar seu HTML diretamente com classes pré-definidas. Para que ele funcione corretamente em um projeto Vite, ele precisa ser processado pelo PostCSS.

CONFIGURAR TAILWIND INTELLISENSE

Para Tailwind CSS v3 e anteriores: A extensão detecta automaticamente o arquivo tailwind.config.{js,cjs,mjs,ts,cts,mts} na raiz do workspace. Se não houver detecção, verifique se o arquivo não está oculto por configurações de exclusão (files.exclude).

Para Tailwind CSS v4 ou projetos com arquivos CSS como entrada: Como o v4 é "CSS first", você deve indicar o caminho do arquivo CSS que importa o Tailwind (ex: @import "tailwindcss") nas configurações do VS Code.

Adicione no settings.json o seguinte:

```
"tailwindCSS.experimental.configFile": "caminho/para/seu/arquivo.css"
```

Além disso, para que as sugestões apareçam dentro de strings (como atributos class="" em JSX ou HTML), é recomendável ajustar a configuração editor.quickSuggestions no settings.json:

```
"editor.quickSuggestions": {
    "strings": "on"
}
```

5.1. O Arquivo `postcss.config.js`

O PostCSS é uma ferramenta para transformar estilos CSS com plugins JavaScript. Ele é essencial para o Tailwind CSS, pois é ele quem processa as classes do Tailwind e adiciona funcionalidades como o `autoprefixer` (que insere prefixos de navegador automaticamente).

Arquivo `postcss.config.js`:

```
export default {
  plugins: {
    // O plugin oficial do Tailwind CSS para PostCSS v4+
    // Ele lê suas classes Tailwind no HTML e gera o CSS correspondente.
    '@tailwindcss/postcss': {},
    // Adiciona prefixos de navegador automaticamente para garantir
compatibilidade
    autoprefixer: {},
  },
};
```

Explicação:

- `@tailwindcss/postcss`: Este é o plugin que permite ao PostCSS entender e compilar as classes do Tailwind CSS. É crucial para o funcionamento do Tailwind em sua versão 4 e posteriores.
- `autoprefixer`: Este plugin escaneia seu CSS e adiciona automaticamente prefixos de fornecedor (`-webkit-`, `-moz-`, etc.) a propriedades CSS que precisam deles para funcionar em navegadores mais antigos ou específicos. Isso garante uma maior compatibilidade do seu design.

5.2. O Arquivo `tailwind.config.js`

Este arquivo é onde você personaliza o Tailwind CSS para se adequar ao design do seu projeto. Aqui, você define as cores, fontes, breakpoints, animações e outros elementos que compõem o seu sistema de design.

Arquivo `tailwind.config.js`:

```
/** @type {import(\'tailwindcss\').Config} */
export default {
    // IMPORTANTE: Onde o Tailwind deve procurar por classes para gerar o CSS.
    // Ele escaneia todos os arquivos HTML e JavaScript/TypeScript para encontrar
as classes usadas.
    content: [
        "./index.html",
        "./src/**/*.{js,ts,jsx,tsx}", // Inclui arquivos JS/TS para classes
dinâmicas
    ],
    theme: {
        extend: {
            // ============================================
            // FONTES PERSONALIZADAS
            // ============================================
            fontFamily: {
                // Define a fonte 'Cinzel' como uma família de fonte customizada
                'cinzel': ['Cinzel', 'serif'],
                // Define a fonte 'Inter' como uma família de fonte customizada
                'inter': ['Inter', 'sans-serif'],
            },

            // ============================================
            // PALETA DE CORES PERSONALIZADA
            // ============================================
            colors: {
                // Define uma escala de tons para a cor 'gold'
                gold: {
                    100: '#F9F1D8',
                    200: '#F0DEAA',
                    300: '#E6CB7D',
                    400: '#DDB850',
                    500: '#D4A532',    // Dourado principal do projeto
                    600: '#B8860B',    // Dourado escuro
                    700: '#8B6914',
                    800: '#5C4510',
                    900: '#2E2208',
                },
                // Define uma escala de tons para a cor 'dark' (preto/cinza
escuro)
                dark: {
                    900: '#0a0a0a',    // Preto principal (fundo)
                    800: '#121212',    // Preto secundário
                    700: '#1a1a1a',    // Usado para backgrounds de cards
                    600: '#2a2a2a',    // Usado para bordas
                }
            },
```

```
            // ============================================
            // ANIMAÇÕES PERSONALIZADAS
            // ============================================
            animation: {
                // Animação para elementos que aparecem de baixo para cima
                'fade-in-up': 'fadeInUp 0.8s ease-out forwards',
                // Animação de fade simples
                'fade-in': 'fadeIn 1s ease-out forwards',
                // Animação para elementos que deslizam da direita
                'slide-in-right': 'slideInRight 0.6s ease-out forwards',
                // Animação de flutuação para elementos decorativos
                'float': 'float 6s ease-in-out infinite',
                // Efeito de brilho para elementos de loading ou destaque
                'shimmer': 'shimmer 2s linear infinite',
            },

            // ============================================
            // KEYFRAMES (DEFINIÇÃO DAS ANIMAÇÕES)
            // ============================================
            keyframes: {
                fadeInUp: {
                    '0%': { opacity: '0', transform: 'translateY(30px)' },
                    '100%': { opacity: '1', transform: 'translateY(0)' },
                },
                fadeIn: {
                    '0%': { opacity: '0' },
                    '100%': { opacity: '1' },
                },
                slideInRight: {
                    '0%': { opacity: '0', transform: 'translateX(50px)' },
                    '100%': { opacity: '1', transform: 'translateX(0)' },
                },
                float: {
                    '0%, 100%': { transform: 'translateY(0)' },
                    '50%': { transform: 'translateY(-20px)' },
                },
                shimmer: {
                    '0%': { backgroundPosition: '-200% 0' },
                    '100%': { backgroundPosition: '200% 0' },
                }
            }
        }
    },
    // Plugins adicionais do Tailwind (ex: para tipografia, formulários, etc.)
    plugins: [],
};
```

Explicação:

- `content`: Esta é a configuração mais importante. Ela informa ao Tailwind quais arquivos ele deve escanear para encontrar as classes que você está usando. Se uma classe não for encontrada nesses arquivos, ela não será incluída no CSS final, otimizando o tamanho do arquivo.

- `theme.extend`: Permite estender o tema padrão do Tailwind com suas próprias configurações, como `fontFamily`, `colors`, `animation` e `keyframes`. Isso evita sobrescrever as configurações padrão e facilita a manutenção.

5.3. O CSS Organizado (`src/style.css`)

O arquivo `src/style.css` é o ponto de entrada para o CSS do seu projeto. Ele deve seguir uma estrutura específica para garantir que o Tailwind CSS funcione corretamente e que seus estilos customizados sejam aplicados na ordem certa.

Arquivo `src/style.css`:

```
/* ============================================
   1. IMPORTS EXTERNOS (SEMPRE NO TOPO!)
   ============================================ */
/* Importa as fontes do Google Fonts para uso no projeto */
@import url('https://fonts.googleapis.com/css2?
family=Cinzel:wght@400;500;600;700&family=Inter:wght@300;400;500;600&display=swap'
);

/* ============================================
   2. DIRETIVAS DO TAILWIND CSS
   ============================================ */
/* Inclui os estilos base do Tailwind (reset CSS, estilos padrão de tags HTML) */
@tailwind base;
/* Inclui os estilos de componentes do Tailwind (classes mais complexas) */
@tailwind components;
/* Inclui as classes utilitárias do Tailwind (flex, p-4, text-red-500, etc.) */
@tailwind utilities;

/* ============================================
   3. ESTILOS BASE CUSTOMIZADOS (@layer base)
   Sobrescreve estilos padrão do navegador ou do Tailwind para tags HTML.
   ============================================ */
@layer base {

  /* Esconde a barra de rolagem padrão em navegadores WebKit (Chrome, Safari) */
  ::-webkit-scrollbar {
    width: 0px;
    background: transparent;
  }

  /* Esconde a barra de rolagem em navegadores Edge/IE e Firefox */
  body {
    -ms-overflow-style: none; /* IE and Edge */
    scrollbar-width: none;    /* Firefox */
    background-color: #0a0a0a; /* Cor de fundo principal do corpo */
    font-family: 'Inter', sans-serif; /* Define a fonte padrão para o corpo */
  }

  /* Suaviza o scroll da página */
  html {
```

```
    scroll-behavior: smooth;
  }
}

/* ============================================
   4. COMPONENTES REUTILIZÁVEIS CUSTOMIZADOS (@layer components)
   Define classes CSS que você usa várias vezes no HTML para elementos mais
complexos.
   ============================================ */
@layer components {

  /* Gradiente dourado para texto, com efeito de clip para que o gradiente apareça
no texto */
  .text-gradient-gold {
    background: linear-gradient(135deg, #F0DEAA 0%, #D4A532 50%, #B8860B 100%);
    -webkit-background-clip: text;
    -webkit-text-fill-color: transparent;
    background-clip: text;
  }

  /* Gradiente dourado para fundo de elementos */
  .bg-gradient-gold {
    background: linear-gradient(135deg, #D4A532 0%, #B8860B 50%, #8B6914 100%);
  }

  /* Borda com gradiente dourado */
  .border-gradient-gold {
    border-image: linear-gradient(135deg, #D4A532, #B8860B) 1;
  }

  /* Efeito de vidro (glassmorphism) */
  .glass-effect {
    background: rgba(26, 26, 26, 0.8);
    backdrop-filter: blur(20px);
    -webkit-backdrop-filter: blur(20px);
  }

  /* Overlay para a seção hero, criando um efeito de escurecimento na imagem de
fundo */
  .hero-overlay {
    background: linear-gradient(to bottom, rgba(10, 10, 10, 0.3) 0%, rgba(10, 10,
10, 0.8) 100%);
  }

  /* Estilos para a animação de revelação no scroll */
  .reveal {
    opacity: 0;
    transform: translateY(30px);
    transition: all 0.8s ease-out;
  }

  .reveal.active {
    opacity: 1;
    transform: translateY(0);
```

```
  }

  /* Efeito de hover em cards */
  .card-hover {
    transition: all 0.4s cubic-bezier(0.4, 0, 0.2, 1);
  }

  .card-hover:hover {
    transform: translateY(-10px);
    box-shadow: 0 20px 40px rgba(212, 165, 50, 0.2);
  }

  /* Animação de entrada do modal */
  .modal-enter {
    animation: modalEnter 0.3s ease-out forwards;
  }

  /* Efeito shimmer para loading/destaque */
  .shimmer-bg {
    background: linear-gradient(90deg, transparent, rgba(255, 255, 255, 0.1),
transparent);
    background-size: 200% 100%;
  }

  /* Correção para texto com gradiente em Safari */
  .gradient-text-fix {
    -webkit-background-clip: text;
    -webkit-text-fill-color: transparent;
    background-clip: text;
    color: transparent;
  }
}

/* ============================================
   5. UTILITIES CUSTOMIZADAS (@layer utilities)
   Define classes CSS pequenas e específicas que podem ser usadas para ajustes
finos.
   ============================================ */
@layer utilities {

  /* Esconde a barra de rolagem em elementos específicos */
  .no-scrollbar::-webkit-scrollbar {
    display: none;
  }

  .no-scrollbar {
    -ms-overflow-style: none;
    scrollbar-width: none;
  }

  /* Sombra de texto para melhorar a legibilidade */
  .text-shadow {
    text-shadow: 0 2px 4px rgba(0, 0, 0, 0.5);
  }
```

```
}

/* ============================================
   6. KEYFRAMES (DEFINIÇÃO DE ANIMAÇÕES)
   Definições de keyframes para as animações customizadas. Ficam fora dos @layer.
   ============================================ */
@keyframes modalEnter {
  from {
    opacity: 0;
    transform: scale(0.9);
  }

  to {
    opacity: 1;
    transform: scale(1);
  }
}

@keyframes float {
  0%,
  100% {
    transform: translateY(0);
  }

  50% {
    transform: translateY(-20px);
  }
}

@keyframes shimmer {
  0% {
    background-position: -200% 0;
  }

  100% {
    background-position: 200% 0;
  }
}

/* ============================================
   7. MEDIA QUERIES E AJUSTES RESPONSIVOS
   Estilos específicos para diferentes tamanhos de tela.
   ============================================ */
@media (max-width: 640px) {
  .hero-title {
    font-size: 2.5rem;
    line-height: 1.2;
  }
}
```

Regras de Ouro para o CSS com Tailwind:

1. **Ordem Importa!**

```
/* SEMPRE NESTA ORDEM: */
@import url(...);     /* 1º - Imports externos (fontes, etc.) */
@tailwind base;       /* 2º - Reset e base do Tailwind */
@tailwind components; /* 3º - Componentes do Tailwind */
@tailwind utilities;  /* 4º - Utilities do Tailwind */

/* Depois seus @layer customizados */
@layer base { ... }
@layer components { ... }
@layer utilities { ... }

/* Por fim, keyframes e media queries globais */
@keyframes ... { ... }
@media ... { ... }
```

2. **O que vai em cada `@layer`?**

Layer	O que colocar	Exemplo
`@layer base`	Reset CSS, estilos de elementos HTML	`body`, `html`, `h1`, scrollbar global
`@layer components`	Classes reutilizáveis grandes	`.card-hover`, `.glass-effect`, `.text-gradient-gold`
`@layer utilities`	Classes pequenas e específicas	`.no-scrollbar`, `.text-shadow`
Fora dos layers	Keyframes, media queries complexas	`@keyframes`, `@media print`

3. **Não duplique!** Evite redefinir classes ou estilos que já estão em um `@layer` ou que o Tailwind já gera. Isso aumenta o tamanho do CSS e pode causar conflitos.

---

6. GUIA DE CORES E DESIGN

O design premium da RW Joias baseia-se em contraste, elegância e uma paleta de cores bem definida para transmitir sofisticação.

6.1. Paleta de Cores

Cor	Código Hexadecimal	Uso Principal
Preto Principal	`#0a0a0a`	Fundo principal da página, elementos escuros.
Preto Secundário	`#121212`	Variações de fundo, cards.
Dourado Principal	`#D4A532`	Destaques, botões, ícones, textos importantes.
Dourado Claro	`#F0DEAA`	Gradientes, detalhes, hover states.

Cor	Código Hexadecimal	Uso Principal
Dourado Escuro	#8B6914	Sombras, bordas, profundidade.
Cinza Claro	#E5E7EB	Textos secundários, detalhes sutis.

Como Aplicar Cores (Exemplos de Classes Tailwind):

```
<!-- FUNDO PRETO -->
<div class="bg-dark-900">...</div>

<!-- TEXTO DOURADO COM GRADIENTE -->
<h1 class="text-gradient-gold">Título Luxuoso</h1>

<!-- BOTÃO DOURADO -->
<button class="bg-gradient-gold text-dark-900">Comprar</button>

<!-- BORDA DOURADA SUTIL -->
<div class="border border-gold-500/20">...</div>

<!-- HOVER DOURADO -->
<a class="hover:text-gold-400 transition-colors">Link</a>
```

6.2. Tipografia e Fontes

A escolha das fontes é fundamental para a identidade visual. Combinamos uma fonte serifada clássica com uma sans-serif moderna para equilíbrio.

Fontes Utilizadas:

Fonte	Uso Principal	Característica
Cinzel	Títulos, Logo	Serifada elegante, estilo clássico luxuoso
Inter	Textos, Parágrafos	Sans-serif moderna, alta legibilidade

Como Usar as Fontes (Exemplos de Classes Tailwind):

```
<!-- TÍTULOS PRINCIPAIS -->
<h1 class="font-cinzel text-5xl font-bold">RW JOIAS</h1>

<!-- SUBTÍTULOS -->
<h2 class="font-cinzel text-3xl font-semibold text-white">Nossa História</h2>

<!-- TEXTOS CORRIDOS -->
<p class="font-inter text-gray-400 leading-relaxed">
  Texto descritivo aqui...
</p>
```

```
<!-- TRACKING (ESPAÇAMENTO ENTRE LETRAS) E CAIXA ALTA (UPPERCASE) -->
<span class="tracking-[0.3em] uppercase">Joalheria</span>
```

6.3. Hierarquia Tipográfica

Manter uma hierarquia clara nos textos ajuda na leitura e na compreensão do conteúdo.

```
<!-- HERO - Título Principal -->
<h1 class="font-cinzel text-5xl md:text-7xl lg:text-8xl font-bold text-white">
  RW <span class="text-gradient-gold">JOIAS</span>
</h1>

<!-- Seção - Título -->
<h2 class="font-cinzel text-4xl md:text-5xl font-bold text-white mb-6">
  Como <span class="text-gradient-gold">Trabalhamos</span>
</h2>

<!-- Card - Título -->
<h3 class="font-cinzel text-xl font-bold text-white mb-3">Consultoria</h3>

<!-- Corpo de texto -->
<p class="font-inter text-gray-400 text-base leading-relaxed">
  Descrição detalhada aqui...
</p>
```

6.4. Boas Práticas de Acessibilidade

Um bom design também é acessível. Algumas dicas:

- **Contraste:** Garanta que a combinação de cores de texto e fundo tenha contraste suficiente para pessoas com deficiência visual. Ferramentas como WebAIM Contrast Checker podem ajudar.
- **Tamanho da Fonte:** Use unidades relativas (`rem`, `em`) para tamanhos de fonte, permitindo que os usuários ajustem o zoom do navegador.
- **Semântica HTML:** Utilize as tags HTML corretamente (`<h1>` para título principal, `<nav>` para navegação, etc.) para que leitores de tela possam interpretar o conteúdo.

---

7. ANATOMIA DO CÓDIGO (HTML, CSS E JS)

A integração correta entre HTML, CSS e JavaScript é o que diferencia um projeto amador de um profissional. Entender como cada parte se conecta é fundamental.

7.1. O HTML Limpo (`index.html`)

No contexto de um projeto Vite, o `index.html` serve como o ponto de entrada principal para o navegador. É crucial que ele seja o mais "limpo" possível, focando na estrutura semântica do conteúdo e delegando a importação de estilos e scripts para o JavaScript.

Estrutura Essencial do `index.html`:

```html
<!DOCTYPE html>
<html lang="pt-BR" class="scroll-smooth"> <!-- `scroll-smooth` para rolagem suave
-->
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>RW JOIAS - O Rei das Jóias</title>

    <!-- Pré-conexão com Google Fonts para carregamento mais rápido -->
    <link rel="preconnect" href="https://fonts.googleapis.com">
    <link rel="preconnect" href="https://fonts.gstatic.com" crossorigin>

    <!-- IMPORTANTE: O CSS NÃO É INCLUÍDO AQUI DIRETAMENTE NO VITE. -->
    <!-- Ele será importado via JavaScript no main.js. -->
</head>
<body class="font-inter text-gray-300 antialiased overflow-x-hidden">
    <!-- Todo o conteúdo visível da sua landing page vai aqui -->
    <!-- Ex: Navbar, Hero, Sobre, Coleções, Galeria, Contato, Footer -->

    <!-- O ponto de entrada do seu JavaScript, que também importará o CSS. -->
    <!-- `type="module"` é essencial para o funcionamento do Vite. -->
    <script type="module" src="/src/main.js"></script>
</body>
</html>
```

Pontos Chave:

- `lang="pt-BR"`: Define o idioma da página, importante para acessibilidade e SEO.
- `scroll-smooth`: Classe do Tailwind que habilita rolagem suave para links âncora.
- `preconnect`: Otimiza o carregamento de recursos externos (como Google Fonts) ao estabelecer uma conexão antecipada.
- `<script type="module" src="/src/main.js"></script>`: Esta é a linha mais importante para a integração com o Vite. Ela carrega seu JavaScript como um módulo ES, e é dentro do `main.js` que o Vite espera encontrar a importação do seu CSS (`import './style.css';`).

7.2. O JavaScript Modular (`src/main.js`)

O `src/main.js` é o cérebro interativo do seu projeto. Ele é responsável por:

- Importar o CSS (conforme o padrão do Vite).
- Gerenciar os dados das coleções e galeria.
- Renderizar dinamicamente o conteúdo na página.
- Implementar todas as funcionalidades interativas (modais, menu mobile, animações).

Estrutura Essencial do `src/main.js`:

```javascript
// 1. Importação do CSS (Obrigatório no Vite para que o Tailwind seja processado)
import './style.css';
```

```
// 2. Dados das Coleções e Galeria (arrays de objetos/strings)
const collections = [ /* ... */ ];
const galleryImages = [ /* ... */ ];

// 3. Funções de Renderização (ex: renderCollections(), renderGallery())
//    Estas funções pegam os dados e injetam o HTML dinamicamente na página.
function renderCollections() {
    const grid = document.getElementById('collections-grid');
    if (grid) {
        grid.innerHTML = collections.map(collection => `<!-- HTML do card --
>`).join('');
    }
}

// 4. Funções de Interatividade (ex: openModal(), closeModal(),
toggleMobileMenu())
//    Estas funções manipulam o DOM em resposta a eventos do usuário.
function openModal(collectionId) {
    // Lógica para encontrar a coleção, preencher o modal e exibi-lo.
    // Garanta que os elementos do DOM existam antes de manipulá-los.
}

function closeModal() {
    // Lógica para esconder o modal e restaurar o scroll do corpo.
}

// 5. Expondo funções para o escopo global (necessário para `onclick` no HTML)
//    Se você usa `onclick="minhaFuncao()"` diretamente no HTML, a função precisa
//    estar acessível globalmente. Caso contrário, use `addEventListener`.
window.openModal = openModal;
window.closeModal = closeModal;

// 6. Listeners de Eventos (ex: scroll, click, keydown)
//    Registra eventos para navbar, menu mobile, fechamento de modal, etc.
document.addEventListener('keydown', (e) => {
    if (e.key === 'Escape') closeModal();
});

// 7. Inicialização (executada quando o DOM está pronto)
document.addEventListener('DOMContentLoaded', () => {
    renderCollections();
    renderGallery();
    // Inicializa o Intersection Observer para as animações de scroll reveal
    document.querySelectorAll('.reveal').forEach(el => {
        observer.observe(el);
    });
});

// 8. Scroll Suave para Links Âncora
document.querySelectorAll('a[href^="#"]').forEach(anchor => {
    anchor.addEventListener('click', function (e) {
        e.preventDefault();
        const target = document.querySelector(this.getAttribute('href'));
        if (target) {
```

```
            target.scrollIntoView({
                behavior: 'smooth',
                block: 'start'
            });
        }
    });
});
```

7.3. Funções de Interatividade (Detalhes)

7.3.1. Modais de Coleção (`openModal`, `closeModal`)

As funções `openModal` e `closeModal` são responsáveis por controlar a exibição do modal de detalhes das coleções. Elas buscam os dados da coleção clicada, preenchem o modal com as imagens e o nome da coleção, e controlam a visibilidade do modal e o scroll do corpo da página.

```
// ABRIR MODAL
function openModal(collectionId) {
    const collection = collections.find(c => c.id === collectionId);
    if (!collection) return; // Garante que a coleção existe

    const modalTitle = document.getElementById('modal-title');
    const modalContent = document.getElementById('modal-content');
    const modal = document.getElementById('collection-modal');

    if (!modalTitle || !modalContent || !modal) return; // Garante que os
elementos existam

    modalTitle.textContent = collection.name;
    modalContent.innerHTML = collection.images.map(img => `
        <div class="aspect-square rounded-lg overflow-hidden bg-dark-700">
            <img src="${img}" alt="${collection.name}" class="w-full h-full
object-cover hover:scale-105 transition-transform duration-500">
        </div>
    `).join('');

    modal.classList.remove('hidden');
    document.body.style.overflow = 'hidden'; // Trava o scroll do corpo
}

// FECHAR MODAL
function closeModal() {
    const modal = document.getElementById('collection-modal');
    if (!modal) return;

    modal.classList.add('hidden');
    document.body.style.overflow = ''; // Libera o scroll do corpo
}

// Fechar modal com a tecla ESC
document.addEventListener('keydown', (e) => {
```

```
    if (e.key === 'Escape') closeModal();
});

// Expor funções globalmente para uso com `onclick` no HTML
window.openModal = openModal;
window.closeModal = closeModal;
```

7.3.2. Animação de Scroll Reveal (`Intersection Observer`)

O `Intersection Observer` é uma API moderna do navegador que permite detectar quando um elemento entra ou sai da viewport. Usamos isso para criar o efeito de *scroll reveal*, onde elementos aparecem suavemente na tela conforme o usuário rola a página.

```
const observerOptions = {
    root: null,        // O viewport é o elemento raiz
    rootMargin: '0px',
    threshold: 0.1     // O callback será executado quando 10% do elemento estiver
visível
};

const observer = new IntersectionObserver((entries) => {
    entries.forEach(entry => {
        if (entry.isIntersecting) {
            entry.target.classList.add('active'); // Adiciona a classe 'active'
para iniciar a animação
            observer.unobserve(entry.target);     // Para de observar após a
primeira animação
        }
    });
}, observerOptions);

// Observa todos os elementos que possuem a classe 'reveal'
document.querySelectorAll('.reveal').forEach(el => {
    observer.observe(el);
});
```

7.3.3. Navbar Sticky Animada

A barra de navegação (`#navbar`) e o botão flutuante de CTA (`#floating-cta`) aparecem e desaparecem suavemente conforme o usuário rola a página, melhorando a experiência de navegação.

```
const navbar = document.getElementById('navbar');
const floatingCta = document.getElementById('floating-cta');

window.addEventListener('scroll', () => {
    const currentScroll = window.pageYOffset;

    if (navbar) {
```

```
        if (currentScroll > 100) { // Se o scroll for maior que 100px
            navbar.classList.remove('opacity-0', '-translate-y-full'); // Mostra a
navbar
        } else {
            navbar.classList.add('opacity-0', '-translate-y-full');    // Esconde
a navbar
        }
    }

    if (floatingCta) {
        if (currentScroll > 100) { // Se o scroll for maior que 100px
            floatingCta.classList.remove('opacity-0', 'translate-y-10'); // Mostra
o CTA
        } else {
            floatingCta.classList.add('opacity-0', 'translate-y-10');    //
Esconde o CTA
        }
    }
});
```

7.3.4. Menu Mobile

O menu mobile é controlado por um botão que alterna a visibilidade do menu, e os links dentro do menu fecham-no automaticamente ao serem clicados.

```
const mobileMenuBtn = document.getElementById('mobile-menu-btn');
const mobileMenu = document.getElementById('mobile-menu');

if (mobileMenuBtn && mobileMenu) {
    mobileMenuBtn.addEventListener('click', () => {
        mobileMenu.classList.toggle('hidden'); // Alterna a classe 'hidden'
    });

    // Fecha o menu mobile quando um link é clicado
    mobileMenu.querySelectorAll('a').forEach(link => {
        link.addEventListener('click', () => {
            mobileMenu.classList.add('hidden');
        });
    });
}
```

7.3.5. Envio de Formulário com Feedback

O formulário de contato (`#contact-form`) possui uma lógica simples para simular o envio e fornecer feedback visual ao usuário.

```
const contactForm = document.getElementById('contact-form');
if (contactForm) {
```

```
    contactForm.addEventListener('submit', (e) => {
        e.preventDefault(); // Previne o comportamento padrão de recarregar a
página

        const btn = e.target.querySelector('button[type="submit"]');
        if (!btn) return;

        const originalText = btn.innerHTML;
        btn.innerHTML = '<span>Mensagem Enviada!</span><svg class="w-5 h-5"
fill="none" stroke="currentColor" viewBox="0 0 24 24"><path stroke-linecap="round"
stroke-linejoin="round" stroke-width="2" d="M5 13l4 4L19 7"/></svg>';
        btn.classList.add('bg-green-600'); // Muda a cor do botão para verde
        btn.classList.remove('bg-gradient-gold'); // Remove o gradiente dourado

        setTimeout(() => {
            btn.innerHTML = originalText; // Restaura o texto original
            btn.classList.remove('bg-green-600');
            btn.classList.add('bg-gradient-gold'); // Restaura o gradiente
            e.target.reset(); // Limpa o formulário
        }, 3000); // Volta ao normal após 3 segundos
    });
}
```

---

8. COMO EDITAR E PERSONALIZAR

Esta seção é dedicada a te guiar na personalização do projeto. Tente fazer as seguintes alterações para praticar e entender melhor como tudo funciona.

8.1. Trocar Imagens

As imagens das coleções e da galeria são definidas no arquivo `src/main.js`. Você tem duas opções para trocá-las:

Opção 1: Usar URLs Externas (Mais fácil para testes)

```
// No src/main.js, dentro dos arrays 'collections' e 'galleryImages'
const collections = [
    {
        id: 'pulseira',
        name: 'Pulseiras',
        image: 'https://seusite.com/nova-imagem-capa.jpg', // Nova URL da imagem
de capa
        images: [
            'https://seusite.com/nova-imagem-1.jpg',
            'https://seusite.com/nova-imagem-2.jpg'
        ]
    }
];
```

Opção 2: Usar Imagens Locais (Recomendado para produção)

Para usar imagens locais, crie uma pasta `images` dentro de `public/` (ex: `public/images/`). Em seguida, referencie as imagens usando caminhos absolutos a partir da raiz do seu projeto.

```
rw-joias/
├── public/
│   ├── images/
│   │   ├── pulseira-capa.jpg
│   │   ├── pulseira-1.jpg
│   │   └── ...
```

```
// No src/main.js, dentro dos arrays 'collections' e 'galleryImages'
const collections = [
    {
        id: 'pulseira',
        name: 'Pulseiras',
        image: '/images/pulseira-capa.jpg', // Caminho para sua imagem local
        images: [
            '/images/pulseira-1.jpg',
            '/images/pulseira-2.jpg'
        ]
    }
];
```

8.2. Editar Textos

A maioria dos textos da página está diretamente no arquivo `index.html`. Basta localizar as seções e tags (`<h1>`, `<p>`, `<a>`, etc.) e editar o conteúdo.

```
<!-- Hero - Título -->
<h1 class="font-cinzel text-5xl">
    SUA <span class="text-gradient-gold">MARCA</span>
</h1>

<!-- Slogan -->
<p class="text-xl italic">Seu slogan aqui</p>

<!-- Sobre - Parágrafos -->
<p>Edite este texto no HTML para contar a história da sua marca...</p>
```

8.3. Adicionar Nova Coleção

Para adicionar uma nova coleção de joias:

1. **No `src/main.js`**, adicione um novo objeto ao array `collections`:

```
const collections = [
    // ... coleções existentes
    {
        id: 'relogios',           // ID único para a coleção (usado para
abrir o modal)
        name: 'Relógios',          // Nome que será exibido na página e no
modal
        image: '/images/relogio-capa.jpg', // Imagem de capa da coleção
        images: [
            '/images/relogio-1.jpg',
            '/images/relogio-2.jpg',
            '/images/relogio-3.jpg',
            '/images/relogio-4.jpg',
            '/images/relogio-5.jpg',
            '/images/relogio-6.jpg'
        ] // Array de imagens para o modal da coleção
    }
];
```

2. **Opcional:** Se você quiser que a nova coleção apareça no menu de navegação, adicione um link correspondente no `index.html`.

O grid de coleções será atualizado automaticamente graças à função `renderCollections()` no `main.js`.

8.4. Mudar Cores para Outros Negócios

O projeto é flexível o suficiente para ser adaptado a outros nichos. Para mudar a paleta de cores, edite o arquivo `tailwind.config.js`.

```
// No tailwind.config.js, dentro de `theme.extend.colors`:
colors: {
    // Exemplo: Para uma Barbearia (Azul e Preto)
    primary: {
        500: '#0066CC',  // Azul principal
        600: '#004499',
    },
    secondary: {
        500: '#CCCCCC',  // Prata/Cinza
    },
    // Remova ou renomeie a paleta 'gold' e 'dark' conforme necessário
}

// Exemplo: Para uma Hamburgueria (Vermelho e Amarelo)
colors: {
    primary: {
        500: '#E63946',  // Vermelho
        600: '#D62828',
    },
    secondary: {
        500: '#F4A261',  // Mostarda/Laranja
```

```
    },
}
```

Lembre-se de que, ao mudar as cores no `tailwind.config.js`, você também precisará ajustar as classes CSS no seu `index.html` e `src/style.css` para usar as novas cores (ex: `bg-primary-500` em vez de `bg-gold-500`).

---

9. GITHUB E VERSIONAMENTO

Dominar o Git e o GitHub é uma habilidade fundamental para qualquer desenvolvedor, especialmente para quem busca uma vaga Júnior. Ele permite controlar versões do seu código, colaborar com outros e manter um histórico de todas as alterações.

9.1. Fluxo Básico do Git

Se você ainda não inicializou um repositório Git para este projeto:

1. **Inicializar o repositório local:** Abra o terminal na pasta raiz do projeto (`rw-joias`) e execute:

   ```
   git init
   ```

2. **Adicionar os arquivos ao stage (área de preparação):**

   ```
   git add .
   ```

   Este comando adiciona todos os arquivos modificados ou novos ao próximo commit.

3. **Criar um commit (salvar um ponto no histórico):**

   ```
   git commit -m "feat: projeto inicial rw-joias"
   ```

   A mensagem do commit (`-m`) deve ser clara e descritiva sobre as alterações feitas.

4. **Conectar ao GitHub e enviar:**

   - Crie um novo repositório vazio no GitHub (sem `README.md`, `.gitignore` ou licença).
   - No terminal, conecte seu repositório local ao remoto e envie o código:

     ```
     git branch -M main
     git remote add origin https://github.com/SEU-USUARIO/SEU-REPOSITORIO.git
     git push -u origin main
     ```

Substitua `SEU-USUARIO` e `SEU-REPOSITORIO` pelos seus dados.

Commits Regulares:

É uma boa prática fazer commits pequenos e frequentes. Após cada alteração significativa:

```
git add .
git commit -m "✨ feat: adicionada seção de depoimentos"
git push origin main
```

9.2. Convenção de Commits

Utilizar uma convenção de commits padroniza as mensagens e facilita a leitura do histórico do projeto. Uma convenção popular é a Conventional Commits.

Exemplos de Tipos de Commit:

Emoji	Tipo	Descrição
✨	`feat`	Nova funcionalidade
🐛	`fix`	Correção de bug
💄	`style`	Mudanças de estilo (formatação, CSS)
📚	`docs`	Alterações na documentação
♻️	`refactor`	Refatoração de código (sem mudança de feature)
⚡	`perf`	Melhoria de performance
🧪	`test`	Adição ou correção de testes
⚙️	`chore`	Tarefas de manutenção (config, dependências)

Formato: `tipo(escopo): mensagem`

- **Tipo:** (obrigatório) `feat`, `fix`, `docs`, `style`, `refactor`, `test`, `chore`, `perf`.
- **Escopo:** (opcional) Onde a mudança ocorreu (ex: `navbar`, `modal`, `vite-config`).
- **Mensagem:** (obrigatório) Descrição concisa da mudança.

---

10. DEPLOY - COLOCANDO ONLINE

Publicar seu projeto na internet é a etapa final para que outras pessoas possam vê-lo. Existem diversas opções gratuitas e fáceis de usar.

10.1. GitHub Pages

O GitHub Pages é uma ótima opção para hospedar sites estáticos diretamente do seu repositório GitHub.

1. **Instale o pacote `gh-pages`:** Este pacote facilita o deploy para o GitHub Pages. Se ainda não o fez, instale-o como uma dependência de desenvolvimento:

```
npm install -D gh-pages
```

2. **Configure o `package.json`:** Adicione a propriedade `homepage` e os scripts `predeploy` e `deploy` ao seu `package.json`:

```
{
  "name": "rwjoias",
  "version": "1.0.0",
  "private": true,
  "type": "module",
  "homepage": "https://SEU-USUARIO.github.io/SEU-REPOSITORIO-GITHUB/", //
Substitua pelos seus dados
  "scripts": {
    "dev": "vite",
    "build": "vite build",
    "preview": "vite preview",
    "predeploy": "npm run build", // Executa o build antes do deploy
    "deploy": "gh-pages -d dist"  // Envia a pasta 'dist' para o GitHub Pages
  },
  "devDependencies": {
    "autoprefixer": "^10.4.27",
    "@tailwindcss/postcss": "^4.2.2", // Adicionado para Tailwind v4
    "postcss": "^8.5.8",
    "tailwindcss": "^4.2.2",
    "vite": "^8.0.1",
    "gh-pages": "^6.1.0" // Dependência para deploy no GitHub Pages
  }
}
```

3. **Configure o `vite.config.js`:** Certifique-se de que a propriedade `base` no seu `vite.config.js` esteja configurada com o nome do seu repositório, como explicado na Seção 4.

4. **Execute o comando de deploy:**

```
npm run deploy
```

Este comando primeiro fará o build otimizado do seu projeto na pasta `dist/` e, em seguida, enviará o conteúdo dessa pasta para uma branch especial (`gh-pages`) no seu repositório GitHub. Seu site estará online em poucos minutos!

10.2. Netlify (Gratuito)

O Netlify é uma plataforma popular para hospedar sites estáticos, oferecendo integração contínua e deploy automático a partir do seu repositório Git.

Método 1: Deploy via Netlify CLI

1. **Instale o Netlify CLI globalmente:**

   ```
   npm install -g netlify-cli
   ```

2. **Faça login no Netlify:**

   ```
   netlify login
   ```

3. **Implante seu site:**

   ```
   netlify deploy --prod
   ```

   Siga as instruções no terminal para configurar seu site.

Método 2: Integração com Git (Recomendado)

1. **Envie seu código para o GitHub:** Certifique-se de que seu projeto esteja no GitHub (ou GitLab, Bitbucket).
2. **Faça login no Netlify:** Acesse app.netlify.com e faça login.
3. **Crie um novo site:** Clique em "Add new site" -> "Import an existing project" -> "Deploy with GitHub" (ou seu provedor Git).
4. **Selecione seu repositório:** Escolha o repositório `rw-joias`.
5. **Configurações de Build:**
   - **Build command:** `npm run build`
   - **Publish directory:** `dist`
6. Clique em "Deploy site". O Netlify detectará automaticamente o Vite e fará o deploy.

10.3. Vercel (Gratuito)

A Vercel é outra excelente plataforma para deploy de front-ends, conhecida por sua facilidade de uso e integração com frameworks modernos como o Vite.

Método 1: Deploy via Vercel CLI

1. **Instale o Vercel CLI globalmente:**

   ```
   npm i -g vercel
   ```

2. **Faça o deploy:**

   ```
   vercel
   ```

Siga as instruções para vincular seu projeto e fazer o deploy.

Método 2: Integração com Git (Recomendado)

1. **Envie seu código para o GitHub:** Seu projeto deve estar no GitHub.
2. **Faça login no Vercel:** Acesse vercel.com e faça login.
3. **Importe seu projeto:** Clique em "Add New..." -> "Project" -> "Import Git Repository" e selecione seu repositório `rw-joias`.
4. **Configurações de Build:** A Vercel geralmente detecta automaticamente que é um projeto Vite e configura o comando de build (`npm run build`) e o diretório de saída (`dist`) corretamente. Confirme as configurações e clique em "Deploy".

---

11. 🛠️ ONDE ERREI E O QUE FOI MELHORADO (Análise de Erros)

Esta seção é crucial para o seu aprendizado. Ela detalha os problemas encontrados no projeto original, explica por que eles ocorriam e como foram corrigidos, transformando-os em lições valiosas.

11.1. Erro de CSS (PostCSS)

Onde você errou: No arquivo `src/style.css`, havia um bloco `@layer utilities {` que **não foi fechado** corretamente com a chave `}`. Além disso, algumas classes customizadas (como `.bg-gradient-gold`) estavam duplicadas, aparecendo tanto dentro de um `@layer` quanto soltas no arquivo.

Por que dava erro: O PostCSS (a ferramenta que processa o seu CSS, incluindo o Tailwind) é muito rigoroso com a sintaxe. Um bloco CSS não fechado ou uma estrutura inválida faz com que ele não consiga parsear o arquivo, resultando em um erro `CssSyntaxError: Unclosed block` durante o `npm run build` ou `npm run dev`. A duplicação de classes, embora não cause um erro de build, aumenta o tamanho do arquivo CSS e pode levar a comportamentos inesperados devido à especificidade do CSS.

Como foi reparado:

- Todas as chaves (`{}`) dos blocos `@layer` foram fechadas corretamente.
- As classes CSS customizadas duplicadas foram removidas, mantendo apenas a definição dentro do `@layer` apropriado (`@layer components` ou `@layer utilities`).
- O arquivo `src/style.css` foi reorganizado para seguir a Regra de Ouro de Ordem para imports e `@layer`s.

11.2. Importação do CSS no HTML

Onde você errou: No `index.html`, o CSS estava sendo importado da maneira tradicional, usando a tag `<link rel="stylesheet" href="src/style.css">`.

Por que dava erro: Em projetos Vite (especialmente com `type="module"` no script principal), o Vite espera que o CSS seja importado via JavaScript (ex: `import './style.css';` no `main.js`). Isso permite que o Vite otimize o CSS, aplique o Hot Module Replacement (HMR) e processe-o corretamente com o PostCSS/Tailwind. Incluir o CSS via `<link>` no HTML pode fazer com que o Tailwind não seja processado ou que o HMR não funcione para estilos.

Como foi reparado:

- A tag `<link rel="stylesheet" href="src/style.css">` foi removida do `index.html`.
- A linha `import './style.css';` foi adicionada no início do arquivo `src/main.js`, delegando a importação do CSS ao pipeline do Vite.

11.3. Funções de Modal não Acessíveis (Scope Issue)

Onde você errou: No `index.html`, os elementos que abriam o modal usavam atributos `onclick` como `onclick="openModal('id')"`. No entanto, as funções `openModal` e `closeModal` estavam declaradas dentro do `src/main.js` sem serem expostas globalmente.

Por que dava erro: Quando você usa `<script type="module">` (como é o padrão no Vite), todo o JavaScript dentro desse módulo é isolado em seu próprio escopo. Isso significa que funções declaradas dentro de `main.js` não são automaticamente acessíveis a partir do escopo global do navegador (onde o `onclick` do HTML tenta encontrá-las). Consequentemente, clicar nos elementos não disparava as funções do modal.

Como foi reparado: No final do arquivo `src/main.js`, as funções `openModal` e `closeModal` foram explicitamente anexadas ao objeto `window`, tornando-as acessíveis globalmente:

```
window.openModal = openModal;
window.closeModal = closeModal;
```

11.4. Configuração Incompleta do Tailwind v4

Onde você errou: O projeto estava utilizando o Tailwind CSS versão 4.2.2, mas o arquivo `postcss.config.js` não estava configurado para usar o plugin correto para esta versão.

Por que dava erro: O Tailwind CSS passou por mudanças significativas em sua arquitetura na versão 4. Ele não é mais um plugin PostCSS "direto" como nas versões anteriores. Em vez disso, ele requer o pacote `@tailwindcss/postcss` para ser processado corretamente pelo PostCSS. A ausência ou configuração incorreta deste plugin resultava em erros de build e no não processamento das classes Tailwind.

Como foi reparado:

- O pacote `@tailwindcss/postcss` foi instalado como uma dependência de desenvolvimento (`npm install -D @tailwindcss/postcss`).
- O arquivo `postcss.config.js` foi atualizado para incluir o plugin `@tailwindcss/postcss` em sua configuração, garantindo que o Tailwind v4 seja processado corretamente.

11.5. Ordem dos Imports no CSS

Onde você errou: O `@import url(...)` das fontes do Google Fonts estava localizado no meio do arquivo `src/style.css`, após as diretivas `@tailwind`.

Por que dava erro: A especificação do CSS exige que todas as regras `@import` sejam as **primeiras** declarações em um arquivo CSS (com exceção de `@charset`). Quando um `@import` aparece depois de outras regras (como `@tailwind`), o PostCSS e o navegador podem reportar um erro de sintaxe ou simplesmente ignorar a importação, fazendo com que as fontes não sejam carregadas.

Como foi reparado: A linha `@import url(...)` foi movida para o topo do arquivo `src/style.css`, antes de todas as diretivas `@tailwind`.

11.6. Melhorias Visuais Adicionais

Aproveitamos as correções para aplicar algumas melhorias visuais que modernizam o layout e a experiência do usuário, mantendo a estrutura original do seu projeto:

- **Navbar com Sombra:** Adicionada uma sombra sutil (`shadow-lg shadow-gold-500/10`) à barra de navegação para realçar o efeito de vidro e dar mais profundidade.
- **Título do Hero Aprimorado:** O título principal na seção Hero recebeu um emoji (✨) e uma sombra de texto (`drop-shadow-lg`) para maior impacto visual e legibilidade sobre a imagem de fundo.
- **Botões Interativos:** Os botões na seção Hero foram aprimorados com efeitos de escala (`hover:scale-110`) e um `backdrop-blur` para o botão de orçamento, tornando-os mais dinâmicos e convidativos.
- **Indicador de Scroll Animado:** O ícone de seta para baixo na seção Hero agora tem uma animação de pulso (`animate-bounce`) e um efeito de escala ao passar o mouse (`hover:scale-125`), incentivando o usuário a rolar a página.

---

12. PRÓXIMOS PASSOS E IDEIAS PARA EVOLUIR

Para continuar seu aprendizado e aprimorar este projeto, aqui estão algumas ideias e sugestões:

1. **Implementar Dark/Light Mode:** Adicione um botão para alternar entre um tema escuro e um tema claro, utilizando variáveis CSS ou classes Tailwind dinâmicas. Isso é uma ótima prática de acessibilidade e personalização.
2. **Filtros de Coleção Dinâmicos:** Crie botões ou um menu suspenso para filtrar as joias por tipo (pulseiras, colares, anéis) sem recarregar a página, utilizando JavaScript para manipular a exibição dos elementos.
3. **Integração com WhatsApp:** Adicione um botão flutuante que, ao ser clicado, abre o WhatsApp com uma mensagem pré-preenchida para solicitar um orçamento rápido. Isso melhora a conversão e a comunicação com o cliente.
4. **Otimização de Imagens:** Explore ferramentas como Cloudinary ou TinyPNG para otimizar as imagens do projeto, reduzindo o tempo de carregamento e melhorando a performance geral.
5. **Adicionar Testes Automatizados:** Comece a escrever testes unitários para as funções JavaScript mais importantes (ex: `openModal`, `closeModal`). Ferramentas como Vitest são excelentes para projetos Vite.
6. **Refatorar JavaScript:** Considere organizar o `main.js` em módulos menores (ex: `modal.js`, `navbar.js`, `animations.js`) e importá-los no `main.js`. Isso melhora a legibilidade e a manutenção do código.
7. **SEO Básico:** Adicione meta tags (`<meta name="description" content="...">`, `<meta name="keywords" content="...">`) e otimize os atributos `alt` das imagens para melhorar a visibilidade do site em motores de busca.
8. **Animações Avançadas:** Explore bibliotecas de animação JavaScript como GSAP ou Framer Motion para criar efeitos visuais ainda mais sofisticados.

Desenvolvido com dedicação para impulsionar sua carreira como Desenvolvedor Júnior. Continue praticando e explorando! 💎 ✨

Autor: David Gilmur - dgilmourcode **Data da Última Atualização:** 03 de Abril de 2026